



学 院 计算机科学与工程学院

学 号 _____

姓 名 _____

指导教师 _____

时 间 _____

实验3 线性回归算法实现及应用

实验目的:

- 1) 强化对线性回归算法原理的理解
- 2) 掌握线性回归算法的应用
- 3) 熟悉梯度下降算法的原理和实现

实验内容:

- 1) 使用给定样本训练单变量线性回归模型，观察训练损失并对测试数据进行预测。
- 2) 根据温度和小花数量样本建立回归模型，预测指定温度下的小花数量。
- 3) 在相同数据和初始参数下，比较较大、合适和较小学习率对梯度下降过程的影响。

算法原理:

单变量线性回归模型表示为 $\hat{y} = wx + b$ ，其中 w 和 b 分别为模型权重和偏置。训练时以均方误差作为损失函数：

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2.$$

梯度下降根据损失对参数的梯度迭代更新参数。学习率控制每次更新的步长；步长过大可能使损失震荡或发散，步长过小则会降低收敛速度。

Ex1 使用实验要求给出的训练样本训练单变量线性回归模型，并输出测试样本预测值。

Ex1 源代码:

```
""" 3-1 """

from pathlib import Path

import matplotlib

#
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
from torch import nn, optim

#
plt.rcParams["font.family"] = "sans-serif"
plt.rcParams["font.sans-serif"] = [
    "Hiragino Sans GB",
```

```

    "Arial Unicode MS",
    "Songti SC",
    "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False

def main():
    #
    torch.manual_seed(7)

    #
    #                                     nn.Linear
    training_input_values = np.array(
        [[3.3], [4.4], [5.5], [6.71], [6.93], [4.168], [9.779], [6.182],
         [7.59], [2.167], [7.042], [10.791], [5.313], [7.997], [3.1]],
        dtype=np.float32,
    )
    training_target_values = np.array(
        [[1.7], [2.86], [2.09], [3.19], [1.694], [1.573], [3.366], [2.596],
         [2.53], [1.221], [2.827], [3.465], [1.65], [2.904], [1.3]],
        dtype=np.float32,
    )
    test_input_values = np.array(
        [[3.0], [4.0], [5.0], [6.0], [7.0], [8.0]], dtype=np.float32
    )
    test_target_values = np.array(
        [[1.5], [2.1], [2.0], [2.5], [2.3], [3.2]], dtype=np.float32
    )

    #   NumPy           PyTorch
    training_input_tensor = torch.from_numpy(training_input_values)
    training_target_tensor = torch.from_numpy(training_target_values)
    test_input_tensor = torch.from_numpy(test_input_values)

    # nn.Linear(1, 1)           1           1           y = wx + b
    linear_model = nn.Linear(1, 1)
    loss_function = nn.MSELoss() #
    learning_rate = 0.001
    optimizer = optim.SGD(linear_model.parameters(), lr=learning_rate)

    number_of_epochs = 2000
    training_loss_history = [] #
    for epoch_number in range(number_of_epochs):
        #
        predicted_training_values = linear_model( training_input_tensor )
        training_loss = loss_function (
            predicted_training_values , training_target_tensor
        )

```

```

#
optimizer.zero_grad()
training_loss.backward()
optimizer.step()

# .item() Python
training_loss_history.append(training_loss.item())

# eval() no_grad()
linear_model.eval()
with torch.no_grad():
    test_prediction_tensor = linear_model(test_input_tensor)
    final_training_loss = loss_function(
        linear_model(training_input_tensor), training_target_tensor
    ).item()

# item()
model_weight = linear_model.weight.item()
model_bias = linear_model.bias.item()
bias_sign = "+" if model_bias >= 0 else "-"
print("3-1")
print(
    f" {len(training_input_values)} "
    f" {number_of_epochs} {learning_rate}"
)
print(
    f" = {model_weight:.6f} × "
    f" {bias_sign} {abs(model_bias):.6f}"
)
print(f"MSE {final_training_loss:.6f}")
print("")
for input_value, actual_value, predicted_value in zip(
    test_input_values[:, 0],
    test_target_values[:, 0],
    test_prediction_tensor[:, 0],
):
    print(
        f"={input_value:.1f}={actual_value:.3f}"
        f"={predicted_value.item():.3f}"
    )

# EXP_3/assets
output_directory = Path(__file__).resolve().parents[1] / "assets"
output_directory.mkdir(parents=True, exist_ok=True)

#
figure, plot_axes = plt.subplots(1, 2, figsize=(10, 4))
epoch_numbers = np.arange(1, number_of_epochs + 1)
plot_axes[0].plot(epoch_numbers, training_loss_history, color="#285f9e")
plot_axes[0].set(

```

```

        title = " ", xlabel=" ", ylabel=" "
    )
    plot_axes[0].grid(alpha=0.25)

    plot_axes[1].scatter(
        training_input_values[:, 0],
        training_target_values[:, 0],
        label=" ",
        color="#285f9e",
    )
    plot_axes[1].scatter(
        test_input_values[:, 0],
        test_target_values[:, 0],
        label=" ",
        color="#d66a3a",
        marker="x",
    )
    line_input_values = np.linspace(
        min(training_input_values[:, 0].min(), test_input_values[:, 0].min()),
        max(training_input_values[:, 0].max(), test_input_values[:, 0].max()),
        100,
    )
    line_prediction_values = model_weight * line_input_values + model_bias
    plot_axes[1].plot(
        line_input_values,
        line_prediction_values,
        label=" ",
        color="#31835b",
    )
    plot_axes[1].set(
        title = " ", xlabel=" ", ylabel=" "
    )
    plot_axes[1].legend()
    plot_axes[1].grid(alpha=0.25)
    figure.tight_layout()
    figure.savefig(output_directory / "ex3_1_regression.png", dpi=180)
    plt.close(figure)
    print("EXP/EXP_3/assets/ex3_1_regression.png")

if __name__ == "__main__":
    main()

```

运行（测试）过程及结果：

固定随机种子后使用 PyTorch 的线性层和随机梯度下降进行 2000 次迭代，学习率为 0.001。得到拟合方程 $\hat{y} = 0.371027x - 0.020065$ ，训练集 MSE 为 0.262173。测试集预测值如下：

x=3.0 -> 1.093 x=4.0 -> 1.464 x=5.0 -> 1.835

$x=6.0 \rightarrow 2.206$ $x=7.0 \rightarrow 2.577$ $x=8.0 \rightarrow 2.948$

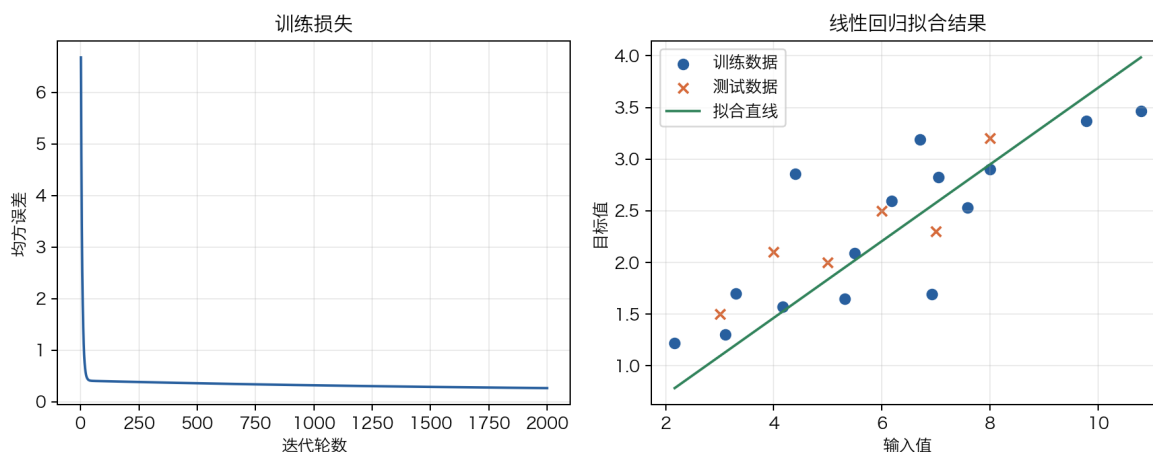


图 1 训练损失变化与线性拟合结果

Ex2 使用温度和小花数量样本训练线性回归模型，预测 18 °C、22 °C 和 33 °C 下的小花数量。

Ex2 源代码：

```

>>>>> 3-2 >>>>>

from pathlib import Path

import matplotlib

#
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
from torch import nn, optim

#
plt.rcParams["font.family"] = "sans-serif"
plt.rcParams["font.sans-serif"] = [
    "Hiragino Sans GB",
    "Arial Unicode MS",
    "Songti SC",
    "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False

def main():
    #

```

```

torch.manual_seed(7)

#
# reshape(-1, 1)
training_temperatures = np.array(
    [15, 20, 25, 30, 35, 40], dtype=np.float32
).reshape(-1, 1)
training_flower_counts = np.array(
    [136, 140, 155, 160, 157, 175], dtype=np.float32
).reshape(-1, 1)
test_temperatures = np.array([18, 22, 33], dtype=np.float32).reshape(-1, 1)

#
temperature_mean = float( training_temperatures .mean())
temperature_standard_deviation = float( training_temperatures .std())
normalized_training_temperatures = (
    training_temperatures - temperature_mean
) / temperature_standard_deviation
normalized_test_temperatures = (
    test_temperatures - temperature_mean
) / temperature_standard_deviation

training_input_tensor = torch.from_numpy(normalized_training_temperatures)
training_target_tensor = torch.from_numpy(training_flower_counts)
test_input_tensor = torch.from_numpy(normalized_test_temperatures)

#
linear_model = nn.Linear(1, 1)
loss_function = nn.MSELoss()
learning_rate = 0.05
optimizer = optim.SGD(linear_model.parameters(), lr=learning_rate)
number_of_epochs = 2000
training_loss_history = []

#
for epoch_number in range(number_of_epochs):
    predicted_training_counts = linear_model( training_input_tensor )
    training_loss = loss_function (
        predicted_training_counts , training_target_tensor
    )
    optimizer.zero_grad()
    training_loss.backward()
    optimizer.step()
    training_loss_history.append( training_loss.item())

#
linear_model.eval()
with torch.no_grad():
    test_prediction_tensor = linear_model( test_input_tensor )
    predicted_test_counts = test_prediction_tensor.numpy().reshape(-1)

```

```

        fitted_training_counts = linear_model( training_input_tensor )
        final_training_mse = loss_function (
            fitted_training_counts , training_target_tensor
        ).item()

#         "
#
normalized_model_weight = linear_model.weight.item()
model_weight = normalized_model_weight / temperature_standard_deviation
model_bias = linear_model.bias.item() - model_weight * temperature_mean

#  $R^2 = 1 - \frac{\text{residual\_sum\_of\_squares}}{\text{total\_sum\_of\_squares}}$ 
residual_sum_of_squares = float (
    (( fitted_training_counts - training_target_tensor ) ** 2).sum().item()
)
total_sum_of_squares = float (
    (( training_target_tensor - training_target_tensor.mean()) ** 2).sum().item()
)
r_squared = 1.0 - residual_sum_of_squares / total_sum_of_squares

print("    3-2    ")
print(
    f"    {len(training_temperatures)}    "
    f"    {number_of_epochs}    {learning_rate}"
)
print(
    f"    = {model_weight:.6f} ×    "
    f"+ {model_bias:.6f}"
)
print(f"    MSE    {final_training_mse:.6f}     $R^2$     {r_squared:.6f}")
print("    ")
for temperature, flower_count in zip(
    test_temperatures[:, 0], predicted_test_counts
):
    print(f"    {int(temperature)} °C -> {flower_count:.2f}    ")

#         assets
output_directory = Path(__file__).resolve().parents[1] / "assets"
output_directory.mkdir(parents=True, exist_ok=True)

#
figure, plot_axes = plt.subplots(1, 2, figsize=(10, 4))
epoch_numbers = np.arange(1, number_of_epochs + 1)
plot_axes[0].plot(epoch_numbers, training_loss_history, color="#285f9e")
plot_axes[0].set(
    title="", xlabel="", ylabel=""
)
plot_axes[0].grid(alpha=0.25)

line_temperature_values = np.linspace(

```

```

        training_temperatures .min(), training_temperatures .max(), 100
    )
    plot_axes [1]. scatter (
        training_temperatures[:, 0],
        training_flower_counts[:, 0],
        label="",
        color="#285f9e",
    )
    line_flower_count_values = (
        model_weight * line_temperature_values + model_bias
    )
    plot_axes [1]. plot (
        line_temperature_values ,
        line_flower_count_values ,
        label="",
        color="#31835b",
    )
    plot_axes [1]. scatter (
        test_temperatures[:, 0],
        predicted_test_counts ,
        label="",
        color="#d66a3a",
        marker="x",
    )
    for temperature, flower_count in zip(
        test_temperatures[:, 0], predicted_test_counts
    ):
        plot_axes [1]. annotate (
            f"{flower_count:.1 f}",
            (temperature, flower_count),
            xytext=(4, 5),
            textcoords="offset points",
        )
    plot_axes [1]. set (
        title = "",
        xlabel=" °C ",
        ylabel=" ",
    )
    plot_axes [1]. legend()
    plot_axes [1]. grid(alpha=0.25)
    figure . tight_layout ()
    figure . savefig ( output_directory / "ex3_2_flower_prediction.png", dpi=180)
    plt . close ( figure )
    print(" EXP/EXP_3/assets/ex3_2_flower_prediction.png")

if __name__ == "__main__":
    main()

```

运行（测试）过程及结果：

对温度特征进行标准化后训练模型，再将参数换算回摄氏温度对应的线性方程。训练 2000 次后，模型为 $\text{花朵数量} = 1.434285T + 114.390424$ ，训练集 MSE 为 17.803167， $R^2 = 0.893906$ 。

温度	预测小花数量
18 °C	140.21 朵
22 °C	145.94 朵
33 °C	161.72 朵

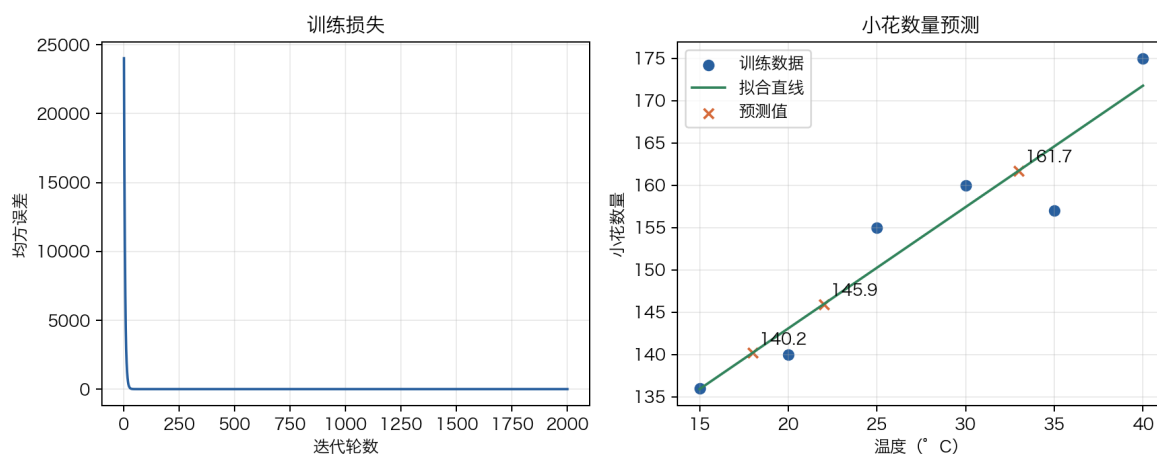


图 2 温度样本、拟合直线和预测结果

Ex3 使用 Ex1 的相同训练样本和初始参数，分别比较较大、合适和较小学习率。

Ex3 源代码：

```

>>>>> 3-3 >>>>>

import math
from pathlib import Path

import matplotlib

#
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
from torch import nn, optim

#
plt.rcParams["font.family"] = "sans-serif"
plt.rcParams["font.sans-serif"] = [
    "Hiragino Sans GB",
    "Arial Unicode MS",

```

```

    "Songti SC",
    "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False

def main():
    # 3-1
    training_input_tensor = torch.tensor(
        [[3.3], [4.4], [5.5], [6.71], [6.93], [4.168], [9.779], [6.182],
         [7.59], [2.167], [7.042], [10.791], [5.313], [7.997], [3.1]],
        dtype=torch.float32,
    )
    training_target_tensor = torch.tensor(
        [[1.7], [2.86], [2.09], [3.19], [1.694], [1.573], [3.366], [2.596],
         [2.53], [1.221], [2.827], [3.465], [1.65], [2.904], [1.3]],
        dtype=torch.float32,
    )

    #
    learning_rate_settings = [
        ("", 0.03),
        ("", 0.01),
        ("", 0.0001),
    ]

    number_of_epochs = 2000
    loss_function = nn.MSELoss()
    line_colors = {
        "": "#d66a3a",
        "": "#31835b",
        "": "#285f9e",
    }

    #
    loss_histories = {}
    training_results = {}

    for learning_rate_description, learning_rate in learning_rate_settings:
        #
        torch.manual_seed(7)
        linear_model = nn.Linear(1, 1)
        optimizer = optim.SGD(linear_model.parameters(), lr=learning_rate)
        loss_history = []
        divergence_iteration = None

        #
        for epoch_index in range(number_of_epochs):
            predicted_values = linear_model(training_input_tensor)
            training_loss = loss_function(predicted_values, training_target_tensor)
            loss_value = training_loss.item()

```

```

#
if not math.isfinite (loss_value):
    divergence_iteration = epoch_index + 1
    break

loss_history.append(loss_value)
optimizer.zero_grad()
training_loss.backward()
optimizer.step()

#                                     MSE
with torch.no_grad():
    final_training_loss = loss_function (
        linear_model( training_input_tensor ), training_target_tensor
    ).item()

loss_histories [ learning_rate_description ] = loss_history
training_results [ learning_rate_description ] = {
    " learning_rate ": learning_rate ,
    " final_loss ": final_training_loss ,
    " divergence_iteration ": divergence_iteration ,
}

print(
    f"      3-3          "
    f"      {number_of_epochs} "
)
for learning_rate_description , learning_rate in learning_rate_settings :
    result = training_results [ learning_rate_description ]
    divergence_iteration = result [" divergence_iteration "]
    if divergence_iteration is None:
        print(
            f" { learning_rate_description } lr={ learning_rate :g} "
            f" MSE={result[' final_loss ']:.6 f}"
        )
    else :
        print(
            f" { learning_rate_description } lr={ learning_rate :g} "
            f" { divergence_iteration } "
            f" MSE={result[' final_loss ']} "
        )

#                                     assets
output_directory = Path(__file__).resolve().parents[1] / "assets"
output_directory.mkdir(parents=True, exist_ok=True)
figure , plot_axis = plt.subplots( figsize =(7, 4.5))

#
for learning_rate_description , learning_rate in learning_rate_settings :

```

```

loss_history = loss_histories [ learning_rate_description ]
if loss_history :
    epoch_numbers = np.arange(1, len( loss_history ) + 1)
    plot_axis . plot(
        epoch_numbers,
        loss_history ,
        label=f"{ learning_rate_description } lr={ learning_rate :g} ",
        color=line_colors [ learning_rate_description ],
    )

plot_axis . set_yscale ("log")
plot_axis . set(
    title ="",
    xlabel="",
    ylabel="",
)
plot_axis . legend()
plot_axis . grid(alpha=0.25, which="both")
figure . tight_layout ()
figure . savefig ( output_directory / "ex3_3_learning_rate .png", dpi=180)
plt . close ( figure )
print(" EXP/EXP_3/assets/ex3_3_learning_rate.png")

if __name__ == "__main__":
    main()

```

运行（测试）过程及结果：

三组实验均最多迭代 2000 次。学习率 0.03 时，损失在第 92 次迭代成为非有限值，训练发散；学习率 0.01 时最终 MSE 为 0.181101；学习率 0.0001 时最终 MSE 为 0.385500，仍在下降但收敛较慢。

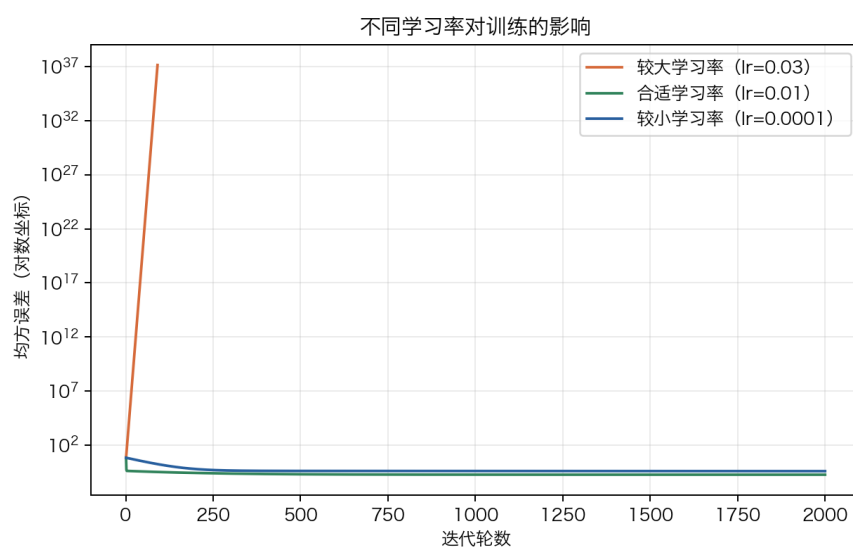


图 3 不同学习率下的训练损失（对数坐标）

总结:

本实验使用 PyTorch 实现了线性回归的前向计算、均方误差计算和梯度下降参数更新。给定的示例数据可以训练出拟合直线，并用于测试样本预测；温度与小花数量样本的回归结果显示两者在本组数据中呈较强线性关系，模型可对指定温度给出近似预测。

学习率实验表明，梯度下降效果不仅取决于迭代次数，也受步长影响。学习率过大时，参数更新越过最优区域并导致损失发散；学习率合适时损失较快下降；学习率较小时训练稳定但在相同迭代预算下尚未充分收敛。温度特征标准化后再训练可以改善参数尺度，训练完成后再把权重换算回原始温度单位，便于解释预测方程。